A Mini Project Report

On

SMART PREDICTIVE MAINTENANCE SYSTEMS USING VIBRATION SENSOR AND ESP32 WITH TINY ML

In partial fulfillment of Requirements for the award of degree
Of

BACHELOR OF TECHNOLOGY

In

ELECTRICAL AND ELECTRONICS ENGINEERING

Submitted by

G. Abhinaya Sri 24RP5A0208

Under the Esteemed Guidance of

B. Sreedhar M Tech

Assistant Professor

Department of Electrical and Electronics Engineering

MEGHA INSTITUTE OF ENGINEERING & TECHNOLOGY FOR WOMEN

(Affiliated to JNTU Hyderabad, Approved by AICTE)

Edulabad(V) , Ghatkesar (M) , Medchal(DIST)-501301

2025-2026

DEPARTMENT OF ELECTRICAL AND ELECTRONICS ENGINEERING

MEGHA INSTITUTE OF ENGINEERING & TECHNOLOGY FOR WOMEN

(Affiliated to JNTU Hyderabad, Approved by AICTE)

Edulabad(V) , Ghatkesar (M) , Medchal(DIST)-501301

(2025 – 2026)

CERTIFICATE

This is to certify that the project work entitled **"Smart Predictive Maintenance System Using Vibration Sensor and ESP32 With Tiny ML"** is submitted by **G. Abhinaya Sri (24RP5A0208).** In partial fulfillment of the requirements for the Degree of Bachelor of Technology in **Electrical & Electronics Engineering** of Megha Institute of Engineering & Technology for Women, during the academic year 2025-26, is a Bonafide record of work carried out under our guidance and supervision. The results embodied in this report have not been submitted to any other University or Institution for the award of any degree.

INTERNAL GUIDE

B. SREEDHAR M Tech

Assistant Professor

HEAD OF DEPARTMENT

P. SANTHOSH KUMAR M Tech

EXTERNAL EXAMINER

PRINCIPAL

DECLARATION

I hereby declare that the project entitled **"SMART PREDICTIVE MAINTANCE SYSTEM USING THE VIBRATION SENSOR WITH TINY ML"** submitted to the JNTUH in the partial Fulfilment of the requirements for the award of the degree of **B. TECH** in **ELECTRICAL AND ELECTRONICS ENGINEERING** is record of an original work done by me under the guidance of **B . SREEDHAR,** Assistant Professor, EEE, Department and this project work has not been submitted any other university for the award of any degree.

Submitted by:

G. Abhinaya Sri 24RP5A0208

ACKNOWLWEGEMENT

It is indeed a great pleasure and immense satisfaction for us to express our deep sense of gratitude, to our project Guide **Mr. B. Sreedhar,** Assistant Professor in Electrical and Electronics Engineering department, for her guidance in the completion of the project.

We convey our sincere gratitude to **Mr. G. SANTHOSH KUMAR**, Head of the department of Electrical & Electronics Engineering for his encouragement in completion of the project.

It is a privilege to express our sincere thanks to **Dr. P. RAMASUBRAMANIAN** Principal, MIETW for extending support and for providing necessary infrastructure and for permitting us to do the project work in this Institute.

We also express our sincere thanks to **Sri. N. MOHAN REDDY, Chairman**, and **Smt. K. MALATHI REDDY, Vice Chairperson** of Megha Institute of Engineering & Technology for Women for their support to go ahead with this project.

Submitted by:

G. Abhinaya Sri 24RP5A0208

ABSTRACT

In many industries, machines and equipment are maintained using traditional methods such as reactive or preventive maintenance. These methods can lead to unexpected machine failures, production delays, increased downtime, and higher maintenance costs. Sudden breakdowns also affect productivity and reduce the lifespan of equipment. Therefore, industries need a system that can continuously monitor machine conditions and predict possible failures in advance. Predictive maintenance systems help solve this problem by analyzing equipment data and allowing maintenance to be performed at the right time, improving efficiency and reliability. Predictive maintenance systems aim to detect machine faults before failure occurs. This project uses a vibration sensor and ESP32 to monitor machine vibrations for predictive maintenance. The collected data is analyzed using Machine Learning to detect abnormal patterns that indicate possible faults.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER-1

INTRODUCTION

1.1. PROBLEM STATEMENT

In modern industries, machinery and equipment are essential for continuous production and efficient operations. However, unexpected equipment failures remain a major challenge, leading to unplanned downtime, increased maintenance costs, reduced productivity, and potential safety hazards. Traditional maintenance approaches, such as reactive maintenance (repair after failure) and preventive maintenance (scheduled servicing), are often inefficient because they either respond too late or perform maintenance unnecessarily.

Many industries still lack cost-effective and real-time monitoring systems that can detect early signs of machine faults. Without continuous monitoring, issues such as bearing wear, imbalance, and misalignment go unnoticed until they cause serious damage. This results in expensive repairs, production losses, and reduced equipment lifespan.

Although advanced predictive maintenance solutions exist, they are often expensive and complex, making them unsuitable for small and medium-scale industries. There is a need for a low-cost, efficient, and easily deployable system that can continuously monitor machine conditions and provide early warnings of potential failures.

This project addresses these challenges by developing a predictive maintenance system using vibration sensors and an ESP32 microcontroller. The system aims to monitor real-time vibration data, detect abnormal patterns, and alert users before equipment failure occurs. By implementing this solution, industries can reduce downtime, optimize maintenance schedules, and improve overall operational efficiency.

1.2.PREDICTIVE MAINTENANCE SYSTEMS

In today's rapidly evolving industrial environment, the reliability and efficiency of machinery play a crucial role in ensuring uninterrupted production and minimizing operational costs. Maintenance strategies have therefore become a key aspect of industrial management. Traditional maintenance approaches, such as reactive maintenance (repairing equipment after failure) and preventive maintenance (servicing equipment at regular intervals), often lead to unnecessary downtime, increased labor costs, and inefficient use of resources. These limitations have driven the need for more intelligent and data-driven maintenance techniques, leading to the development of predictive maintenance (PdM).

Predictive maintenance is a modern approach that utilizes real-time data, sensor technologies, and analytical methods to monitor the actual condition of equipment and predict potential failures before they occur. By continuously analyzing machine parameters such as vibration, temperature, and noise, predictive maintenance systems can identify early warning signs of faults and enable timely intervention. This not only reduces unexpected breakdowns but also extends the lifespan of machinery and improves overall system reliability.

Among various condition monitoring techniques, vibration analysis is one of the most widely used and effective methods for detecting mechanical faults. Rotating machinery such as motors, pumps, compressors, and turbines generate specific vibration signatures during normal operation. Any deviation from these normal patterns can indicate issues such as imbalance, misalignment, looseness, or bearing defects. By capturing and analyzing vibration signals, it is possible to detect these faults at an early stage and prevent catastrophic failures.

The emergence of the Internet of Things (IoT) has further enhanced the capabilities of predictive maintenance systems. IoT-enabled devices allow seamless data collection, communication, and remote monitoring. In this context, microcontrollers like the ESP32 have gained significant popularity due to their low cost, high processing capability, and built-in Wi-Fi and Bluetooth features. The ESP32 enables real-time acquisition of sensor data, on-device processing, and transmission of information to cloud platforms for further analysis and visualization.

This project presents the design and implementation of a predictive maintenance system using a vibration sensor and ESP32 microcontroller. The system continuously monitors vibration levels of machinery, processes the data to detect anomalies, and generates alerts when abnormal conditions are identified. Additionally, the use of wireless communication allows remote monitoring, making the system suitable for industrial and smart applications.

The proposed system aims to provide a cost-effective, scalable, and efficient solution for condition-based monitoring. By adopting such technologies, industries can significantly reduce maintenance costs, improve operational efficiency, enhance safety, and move towards smarter & more automated maintenance practices.

1.3. IMPORTANCE FOR PREDICTIVE MAINTENANCE SYSTEM

- **Prevents Unexpected Equipment Failures**

Predictive maintenance helps detect potential faults in machinery before they lead to breakdowns. By monitoring vibration patterns, the system can alert operators to irregularities, minimizing unplanned downtime and avoiding costly emergency repairs.

- **Reduces Maintenance Costs**

Unlike routine or reactive maintenance, predictive maintenance targets only components that show signs of wear or malfunction. This reduces unnecessary maintenance work and extends the lifespan of machinery.

- **Enhances Safety**

Equipment failures can cause accidents and injuries. By predicting failures early, predictive maintenance improves workplace safety and protects both personnel and machinery.

- **Optimizes Operational Efficiency**

Monitoring vibration and other parameters continuously ensures machines operate at optimal Conditions. This reduces energy waste, improves productivity, and maintains consistent product quality.

- **Data-Driven Decision Making**

Collecting vibration data through sensors and analyzing it via systems like ESP32 enables informed maintenance decisions. Companies can schedule maintenance based on actual machine condition rather than fixed intervals.

- **Supports Smart Manufacturing and Industry 4.0**

Predictive maintenance aligns with modern industrial automation practices. Using IoT devices like ESP32, factories can implement intelligent monitoring systems, contributing to a smart, connected, and efficient production environment.

CHAPTER-2

PREVIOUS WORK

2.1. Edge-Based Predictive maintenance Using Sensor Networks (2020)

A predictive maintenance system was developed using distributed sensor networks and machine learning techniques for equipment monitoring. The system collected vibration and operational data from machines and applied intelligent algorithms to predict failures. It demonstrated that edge-based processing reduces latency and improves real-time decision-making in maintenance systems.

2.2. IoT-Based Machine Condition Monitoring Using ESP32 (2021)*

Researchers designed an IoT-enabled predictive maintenance system using ESP32 and vibration sensors. The system monitored machine conditions continuously and transmitted data over Wi-Fi for cloud-based analysis. Alerts were generated using indicators such as buzzers and LEDs when abnormal vibration patterns were detected, improving early fault detection. Fire and Gas Detection Robot (2017)

2.3. Smart Vibration Monitoring System for Industrial Equipment (2022)*

A smart system was proposed for monitoring machine health using vibration sensors and embedded controllers. The vibration signals were analyzed using frequency-domain techniques such as FFT to detect faults like imbalance and misalignment. The system proved effective in reducing unexpected machine failures and improving maintenance planning.

2.4. Low-Power Predictive Maintenance Using Neural Networks (2024)*

Recent studies focused on using neural networks for vibration-based predictive maintenance in low-power devices. The system processed vibration data using machine learning models to identify patterns of wear and faults. It highlighted that on-device intelligence can reduce dependency on cloud processing while maintaining accuracy.

2.5. Wireless Condition Monitoring System for Industrial Equipment (2025)

A wireless condition monitoring system was developed using ESP32 and vibration sensors to monitor industrial machines remotely. Data was transmitted over Wi-Fi to a central server where analysis was performed. The system reduced manual inspection and allowed continuous monitoring of machine health, improving reliability and safety.

CHAPTER-3

PROJECT EXPLANATION

3.1. BLOCK DIAGRAM

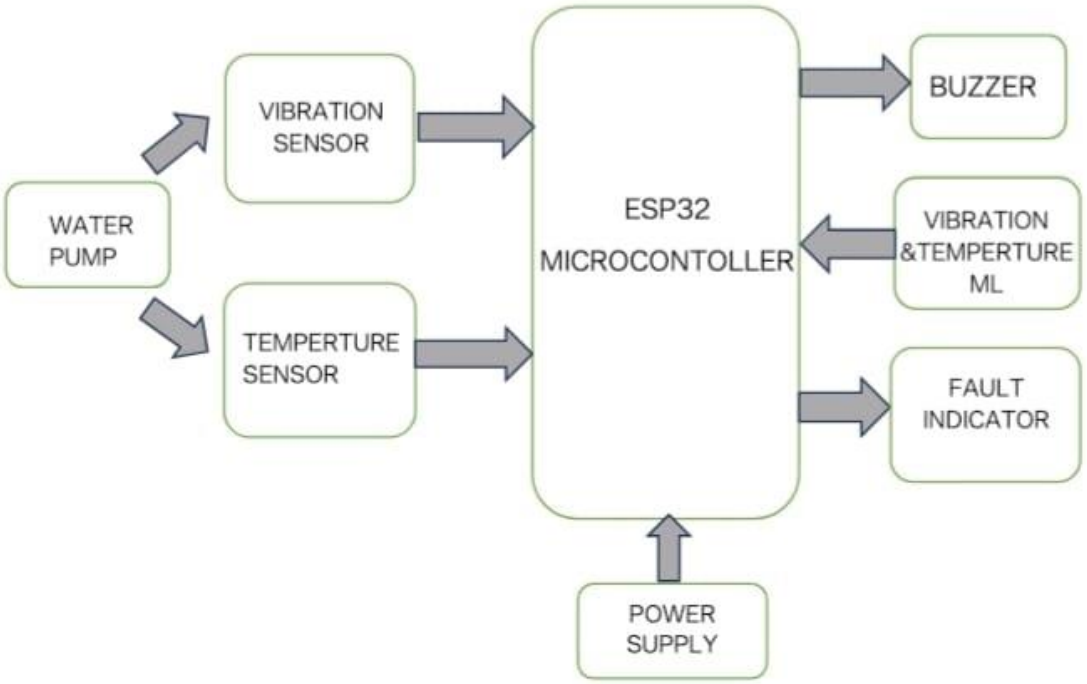


Figure 3.1. Block diagram of Predictive Maintenance System Using Vibration sensor and ESP32 with Tiny ML

The block diagram represents an ESP32-based monitoring and control system for a water pump, designed to detect operational abnormalities and provide alerts.

The system consists of a water pump equipped with a vibration sensor and a temperature sensor that feed real-time data to the ESP32 microcontroller. The ESP32 processes these inputs to assess the pump's condition, analyzing vibration levels and temperature readings. A power supply provides energy to the ESP32 and associated components.

Based on the processed data, the ESP32 controls three outputs: a buzzer that sounds an alarm for faults, a fault indicator that visually signals issues, and a vibration & temperature ML module that handles advanced data analysis or machine-learning-based predictive maintenance.

The overall purpose is to monitor the pump's health, detect faults early through vibration and temperature measurements, and prevent equipment damage or failure by issuing alerts or triggering protective actions.

3.2 PROJECT EXPLANATION AND WORKING

This project focuses on developing a smart predictive maintenance system using a vibration sensor, DHT11 temperature sensor, and ESP32 microcontroller with Machine Learning. The system continuously monitors machine vibration and temperature to detect faults at an early stage. It also provides alerts using a buzzer and LED, making it useful for industrial safety and automation.

The ESP32 acts as the main controller and receives data from both the vibration sensor and the DHT11 sensor. The vibration sensor detects mechanical vibrations, while the DHT11 measures temperature. These sensors convert physical conditions into electrical signals, which are processed by the ESP32.

A simple Machine Learning model values are used to analyze the data. If vibration and temperature values are within the normal range, the system continues monitoring. If any value exceeds the set limit, it indicates an abnormal condition.

When a fault is detected, the ESP32 activates the alert system. The LED turns ON to provide visual indication, and the buzzer sounds to alert the user. This helps in taking immediate maintenance action.

Overall, the system performs real-time monitoring and early fault detection, improving machine efficiency, safety, and reliability.

DESIGN AND IMPLEMENTATION

This project uses two important platform.

1. Software Platform and
2. Hardware Platform.

These platforms are discussed below

3.2.1 Software platform

- Arduino IDE
- Machine Learning
- Language: Micro Python

3.2.2 Hardware platform

. The hardware implementation uses an ESP32 as the main controller. It is connected to a vibration sensor for condition monitoring and a DHT11 sensor for temperature and humidity measurement. A machine learning model processes the sensor data to detect abnormalities. An LED and buzzer are used to provide visual and audio alerts. A water pump is automatically controlled based on system conditions. A regulated power supply ensures smooth operation of all components

CHAPTER-4
SOFTWARE

4.1 INTRODUCTION TO ARDUINO IDE

Arduino is a prototype platform (open-source) based on an easy-to-use hardware and software. It consists of a circuit board, which can be programmed (referred to as a microcontroller) and a ready-made software called Arduino IDE (Integrated Development Environment), which is used to write and upload the computer code to the physical board.

The key features are:

- Arduino boards are able to read analog or digital input signals from different sensors and turn it into an output such as activating a motor, turning LED on/off, connect to the cloud and many other actions.
- You can control your board functions by sending a set of instructions to the microcontroller on the board via Arduino IDE (referred to as uploading software).
- Unlike most previous programmable circuit boards, Arduino does not need an extra piece of hardware (called a programmer) in order to load a new code onto the board. You can simply use a USB cable.
- Additionally, the Arduino IDE uses a simplified version of C++, making it easier to learn to program.
- Finally, Arduino provides a standard form factor that breaks the functions of the micro-controller into a more accessible package.

After learning about the main parts of the Arduino UNO board, we are ready to learn how to set up the Arduino IDE. Once we learn this, we will be ready to upload our program on the Arduino board.

4.2 ARDUINO DATA TYPES

Data types in C refers to an extensive system used for declaring variables or functions of different types. The type of a variable determines how much space it occupies in the storage and how the bit pattern stored is interpreted.

The following table provides all the data types that you will use during Arduino programming.

Void:

The void keyword is used only in function declarations. It indicates that the function is expected to return no information to the function from which it was called.

Example:

```
Void Loop ( )
{
// rest of the code
}
```

Boolean:

A Boolean holds one of two values, true or false. Each Boolean variable occupies one byte of memory.

Example:

```
Boolean state= false ; // declaration of variable with type boolean and initialize it with false.
Boolean state = true ; // declaration of variable with type boolean and initialize it with false.
```

Char:

A data type that takes up one byte of memory that stores a character value. Character literals are written in single quotes like this: 'A' and for multiple characters, strings use double quotes: "ABC".

Example:

```
Char chr_a = ‘a’;//declaration of variable with type char and initialize it with character a.
Char chr_c = 97;//declaration of variable with type char and initialize it with character 97
```

Unsigned char:

Unsigned char is an unsigned data type that occupies one byte of memory. The unsigned char data type encodes numbers from 0 to 255.

Example:

```
Unsigned Char chr_y = 121; // declaration of variable with type Unsigned char , initialize it with character y
```

Byte:

A byte stores an 8-bit unsigned number, from 0 to 255.

Example:

```
Byte m = 25; //declaration of variable with type byte and initialize it with 25
```

Int:

Integers are the primary data-type for number storage. **Int** stores a 16-bit (2-byte) value. This yields a range of -32,768 to 32,767 (minimum value of -2^15 and a maximum value of (2^15) - 1).The **int** size varies from board to board. On the Arduino Due, for example, an **int** stores a 32-bit (4-byte) value. This yields a range of -2,147,483,648 to 2,147,483,647 (minimum value of -2^31 and a maximum value of (2^31) - 1).

Example:

Int counter = 32; // declaration of variable with type int and initialize it with 32.

Unsigned int:

Unsigned ints (unsigned integers) are the same as int in the way that they store a 2 byte value. Instead of storing negative numbers, however, they only store positive values, yielding a useful range of 0 to 65,535 (2^16) - 1). The Due stores a 4 byte (32-bit) value, ranging from 0 to 4,294,967,295 (2^32 - 1).

Example:

Unsigned int counter= 60; // declaration of variable with type unsigned int and initialize it with 60.

Word:

On the Uno and other ATMEGA based boards, a word stores a 16-bit unsigned number. On the Due and Zero, it stores a 32-bit unsigned number.

Example

Word w = 1000; //declaration of variable with type word and initialize it with 1000.

Long:

Long variables are extended size variables for number storage, and store 32 bits (4 bytes), from 2,147,483,648 to 2,147,483,647.

Example:

Long velocity= 102346; //declaration of variable with type Long and initialize it with 102346

Unsigned long:

Unsigned long variables are extended size variables for number storage and store 32 bits (4 bytes). Unlike standard longs, unsigned longs will not store negative numbers, making their range from 0 to 4,294,967,295

Example:

Unsigned Long velocity = 101006; // declaration of variable with type Unsigned Long and initialize it with 101006.

Float:

Data type for floating-point number is a number that has a decimal point. Floating-point numbers are often used to approximate the analog and continuous values because they have greater resolution than integers.Floating-point numbers can be as large as 3.4028235E+38 and as low as 3.4028235E+38. They are stored as 32 bits (4 bytes) of information.

Example:

Float num = 1.352;//declaration of variable with type float and initialize it with 1.352.

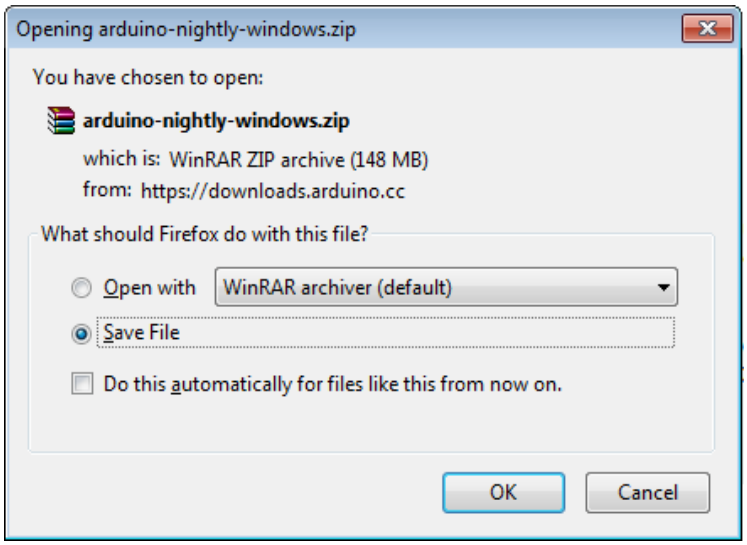
Double:

The double data type stores decimal numbers with higher precision than float. It usually uses 8 bytes (64 bits) of memory and is useful for accurate calculations. In boards like the ESP32, it provides better precision for handling complex numerical data

. **Figure 4.2: Micro USB Cable**

Step 2: Download Arduino IDE Software:

You can get different versions of Arduino IDE from the Download page on the Arduino Official website. You must select your software, which is compatible with your operating system (Windows, IOS, or Linux). After your file download is complete, unzip the file.

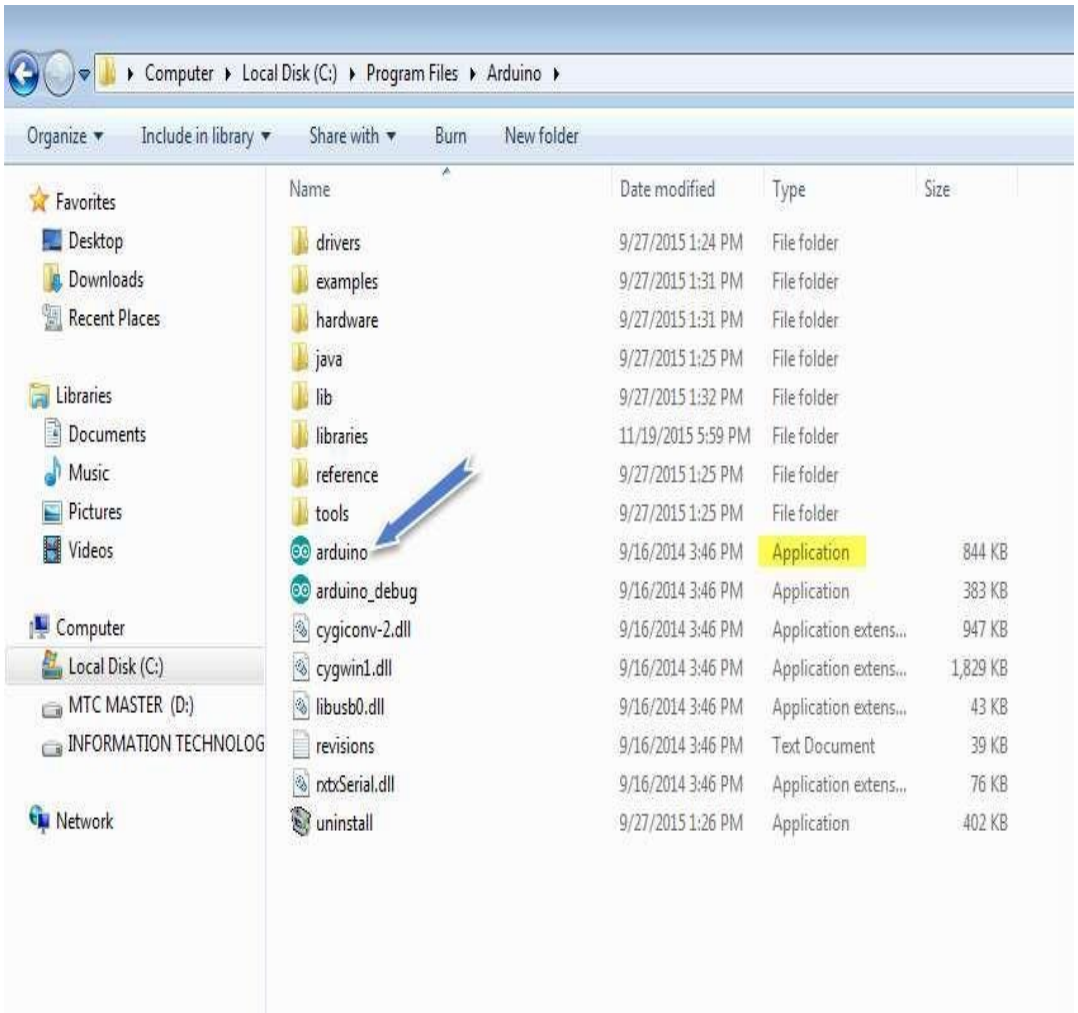


Step 3: Power up your board:

The Arduino Uno, Mega, Duemilanove and Arduino Nano automatically draw power from either, the USB connection to the computer or an external power supply. If you are using an Arduino Diecimila, you have to make sure that the board is configured to draw power from the USB connection. Check that it is on the two pins closest to the USB port. Connect the Arduino board to your computer using the USB cable. The green power LED (labeled PWR) should glow.

Step 4: Launch Arduino IDE:

After your Arduino IDE software is downloaded, you need to unzip the folder. Inside the folder, you can find the application icon with an infinity label (application.exe). Doubleclick the icon to start the IDE.

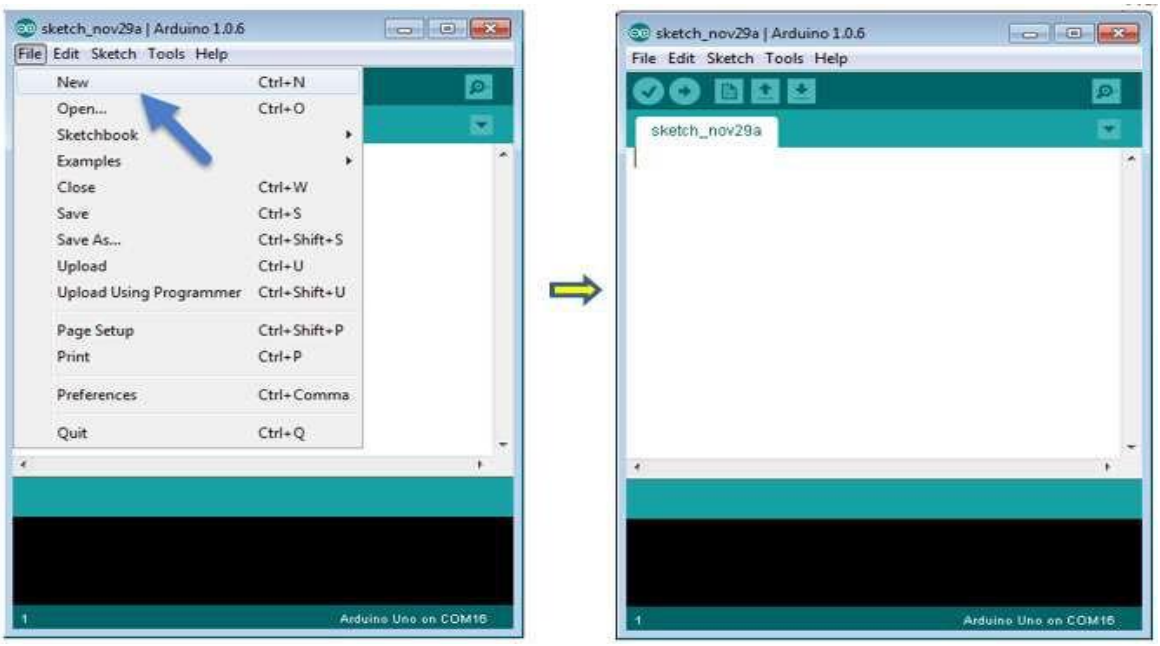


Step 5: Open your first project:

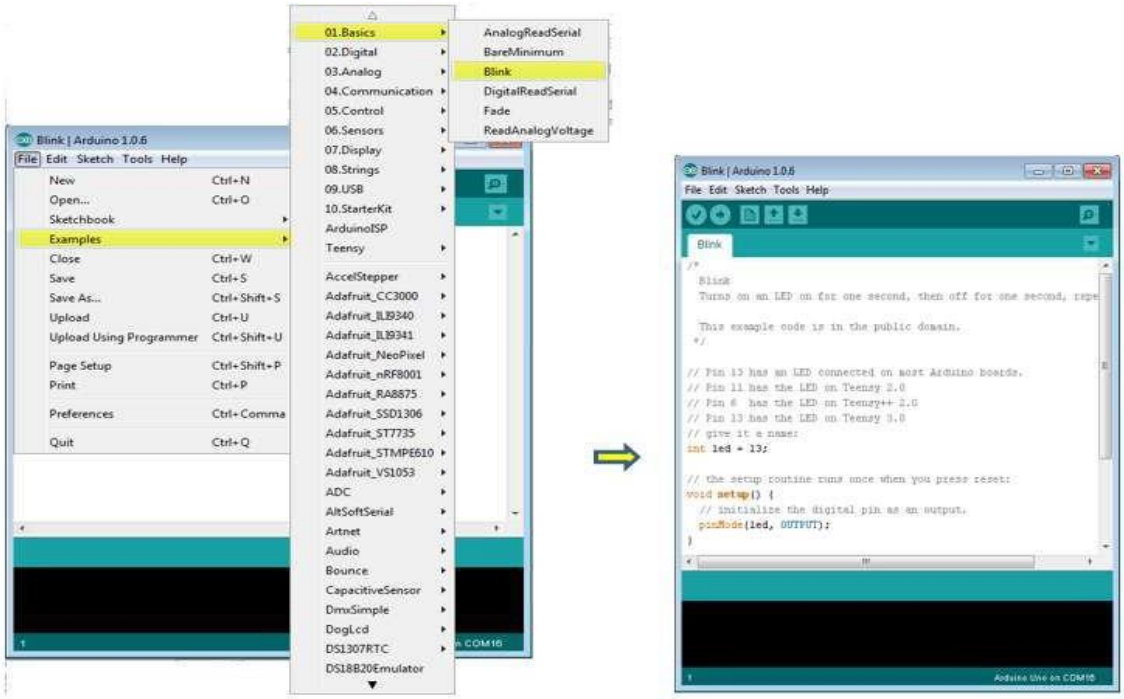
Once the software starts, you have two options:

- Create a new project.
- Open an existing project example.

To create a new project, select File --> New.To open



To open an existing project example, select File -> Example -> Basics -> Blink.

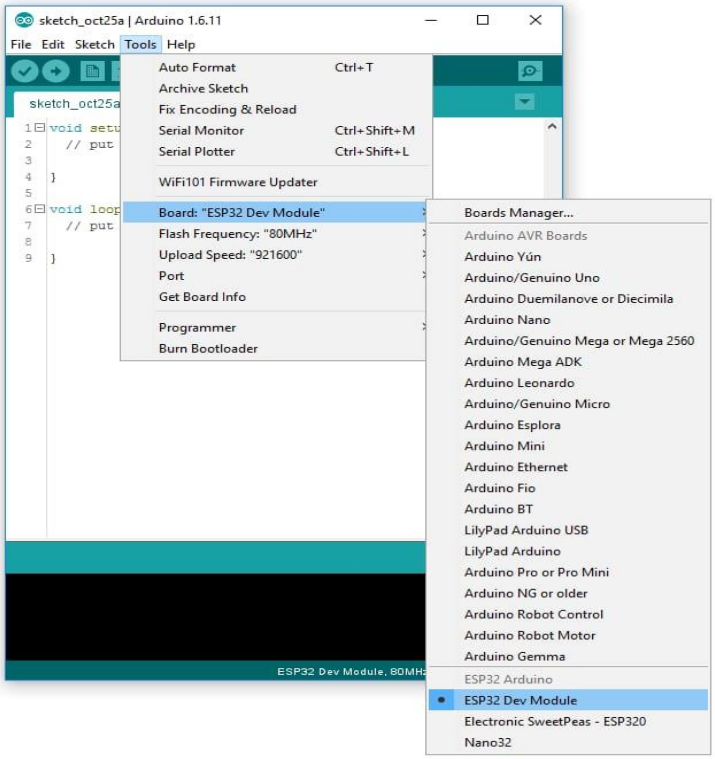


Here, we are selecting just one of the examples with the name **Blink**. It turns the LED on and off with some time delay. You can select any other example from the list.

Step 6: Select your ESP32 DEV board:

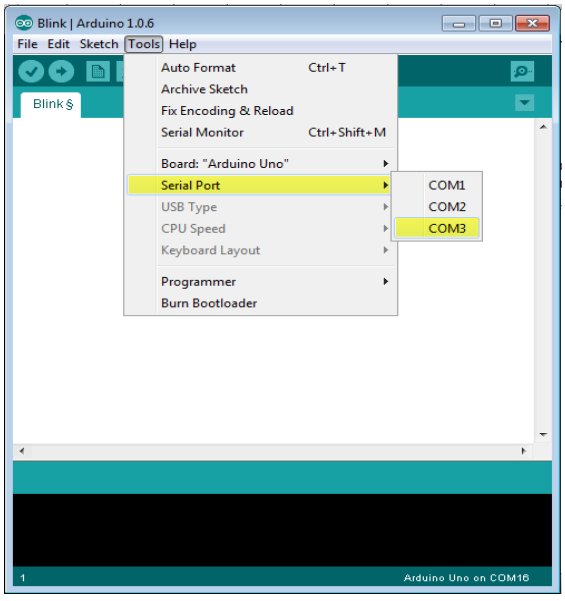
To avoid any error while uploading your program to the board, you must select the correct ESP32 Dev board name, which matches with the board connected to your computer.

Go to Tools -> Board and select your board. Here,we have selected ESP32 Dev board according to our tutorial, but you must select the name matching the board that you are using



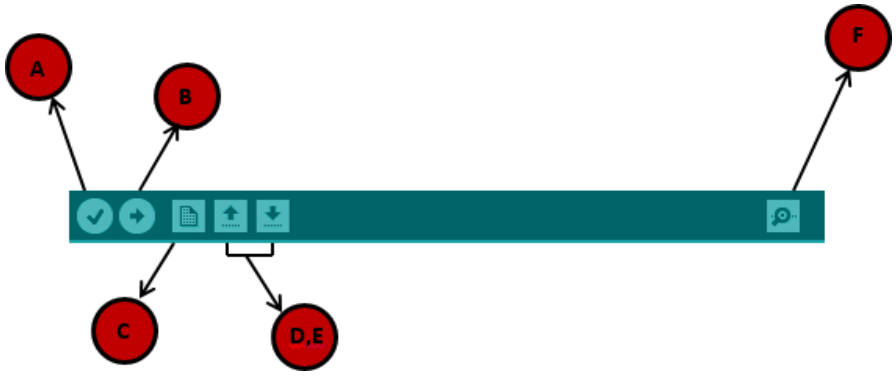
Step 7: Select your serial port:

Select the serial device of the Arduino board. Go to **Tools** ->**Serial Port** menu. This is likely to be COM3 or higher (COM1 and COM2 are usually reserved for hardware serial ports). To find out, you can disconnect your Arduino board and re-open the menu, the entry that disappears should be of the Arduino board. Reconnect the board and select that serial port.



Step 8: Upload the program to your board:

Before explaining how we can upload our program to the board, we must demonstrate the function of each symbol appearing in the Arduino IDE toolbar.



A- Used to check if there is any compilation error.

B- Used to upload a program to the ESP32 board.

C- Shortcut used to create a new sketch.

D- Used to directly open one of the example sketch.

E- Used to save your sketch.

F- Serial monitor used to receive serial data from the board and send the serial data to the board.

Now, simply click the "Upload" button in the environment. Wait a few seconds; you will see the RX and TX LEDs on the board, flashing. If the upload is successful, the message "Done uploading" will appear in the status bar.

4.3 ESP32 PROGRAMMING STRUCTURE

In this chapter, we will study in depth, the Esp32 program structure and we will learn more new terminologies used in the ESP32 world. The Arduino ide software is open-source. The source code for the Java environment is released under the GPL and the C/C++ microcontroller libraries are under the LGPL.

Sketch:

The first new terminology is the ESP32 program called "**sketch**".

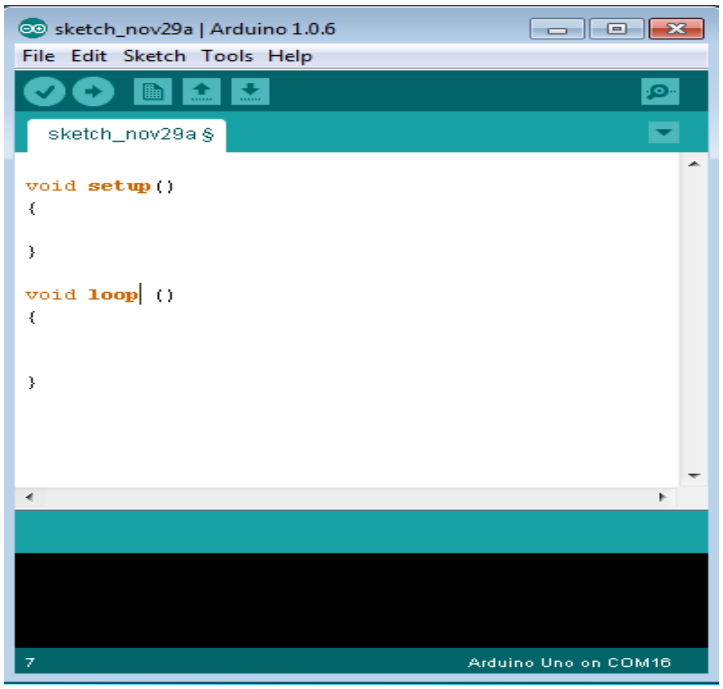
Structure:

Esp32 programs can be divided in three main parts: Structure, Values (variables and constants), and Functions. In this tutorial, we will learn about the Arduino IDE software program, step by step, and how we can write the program without any

syntax or compilation error.

Let us start with the Structure. Software structure consists of two main functions:

- Setup( ) function
- Loop( ) function



Void setup ( )

{

}

PURPOSE:

The **setup ()** function is called when a sketch starts. Use it to initialize the variables, pin modes, start using libraries, etc. The setup function will only run once, after each power up or reset of the Esp32 board.

INPUT

OUTPUT

RETURN

4.4. TINY ML

Tiny ML (Tiny Machine Learning) is a specialized field of machine learning that focuses on running intelligent models on small, resource-constrained devices such as microcontrollers, sensors, and embedded systems like the ESP32. Unlike traditional ML systems that depend on cloud computing, TinyML performs data processing directly on the device (edge computing), enabling faster response and reduced dependency on internet connectivity.

TinyML models are first trained on powerful computers using large datasets. Then, they are compressed, optimized, and converted into lightweight formats so they can fit into devices with limited memory, low processing power, and minimal energy consumption. These models can perform tasks like classification, prediction, and anomaly detection in real time.

Key Features of Tiny ML:-

- **Low Power Consumption** – Operates on very low energy, suitable for battery-powered devices
- **Edge Intelligence** – Processes data locally on the device without sending it to the cloud
- **Real-Time Performance** – Provides quick responses with minimal delay
- **Small Memory Footprint** – Models are highly optimized to fit into limited storage (KBs to MBs)
- **Low Latency** – Faster decision-making since data doesn't travel over networks
- **Cost-Effective** – Reduces need for expensive cloud infrastructure
- **Enhanced Privacy & Security** – Sensitive data remains on the device
- **Scalability** – Can be deployed across many small devices in IoT systems

4.4.1. PROCESS TO TRAIN THE MODEL:

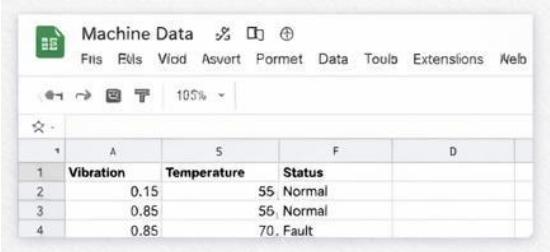
Here is a simple step-by-step process to train a TinyML model using a Google Sheet dataset in Google Colab.

Step 1:- Prepare Dataset in Google Sheets

- **Create a dataset in Google Sheets**

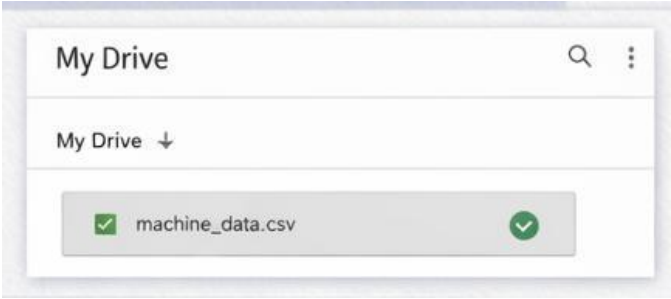
Example columns:

- **Vibration value**
- **Temperature**
- **Status (Normal / Fault)**
- **Save and clean the data (no empty cells)**



Step 2:- Upload to Google Drive

- Go to Google Drive
- Download your sheet as **CSV file**
- Upload the CSV file to Drive

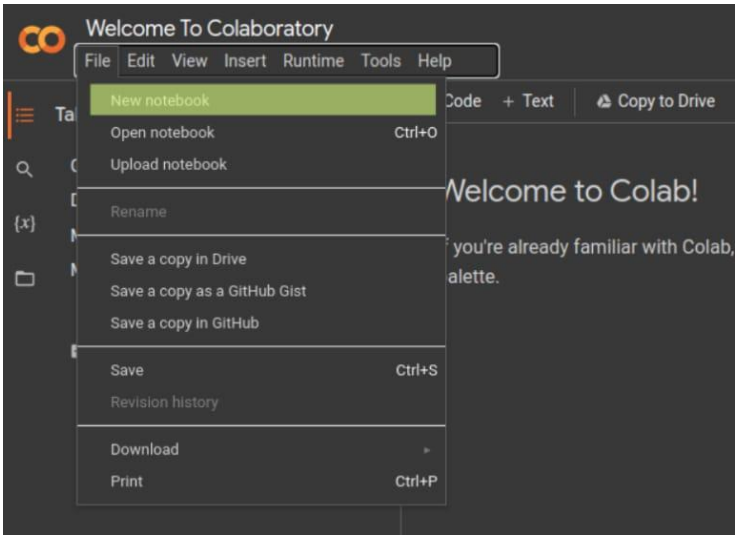


Step 3:-Setting up Google Colab

Since Google Colab runs in the cloud, there's no installation required. All you need is a Google account.

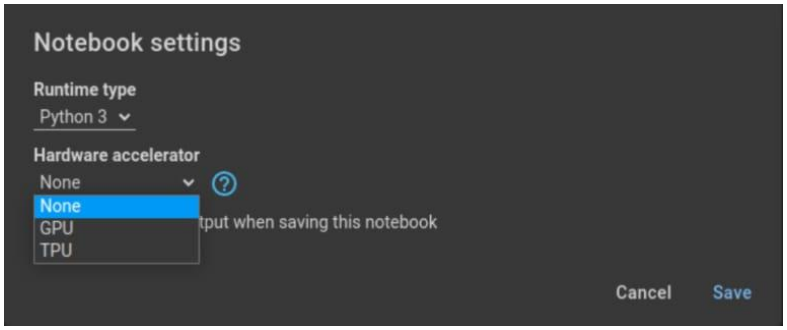
Here's how to get started:

1. **Open Google Colab**: Go to Google Colab and sign in with your Google account.
2. **Create a New Notebook:** Once you're on the Google Colab interface, click on File > New notebook to create a new notebook.



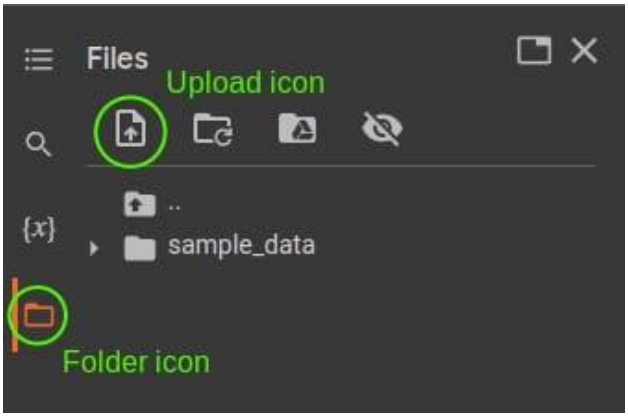
3. **Change the Runtime Type:**

For deep learning, you'll want to utilize the power of a GPU. Click on Runtime > Change runtime type, and select GPU from the Hardware Accelerator drop-down menu.

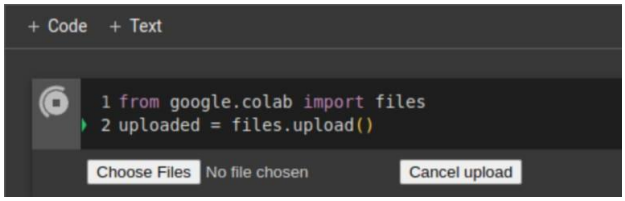


Step 4:-Uploading Files to google Colob:

1. Before we dive into writing deep learning code, let's talk about how to upload files to Google Colab, which you might need for training models.
2. Use the File Browser: On the left-hand side, click on the folder icon to open the file browser. You can upload files by clicking on the upload icon.

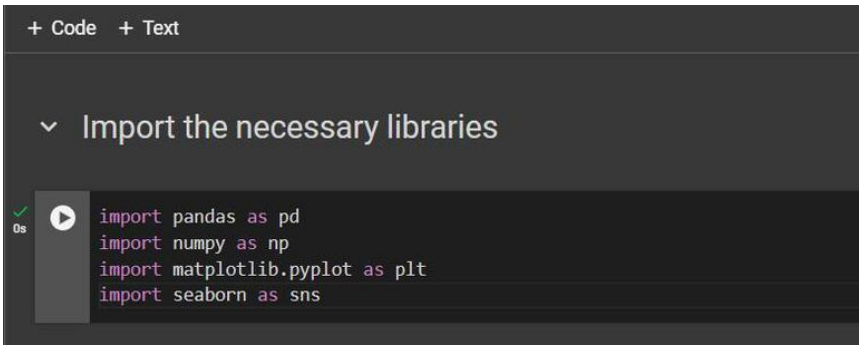


Step 5:- Use +code



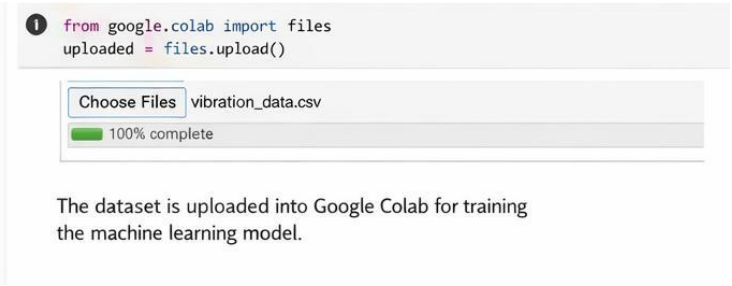
Step 6:- Import Libraries

1. Essential libraries are imported for data processing, visualization, and machine learning model development.
2. After installing the necessary libraries, Whether you're using pandas for machine learning to define, compile, and you can easily train your model.

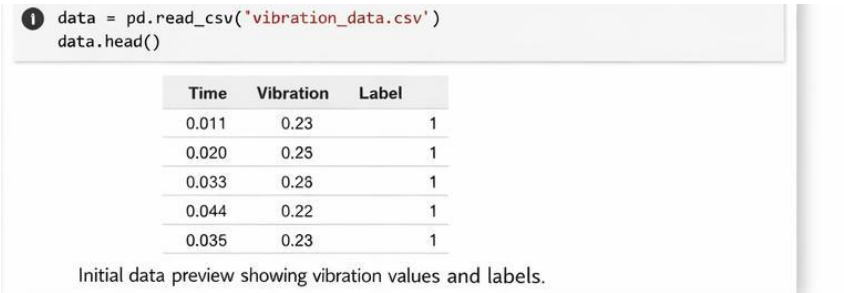


Step 7 :- Load Data Sets

The dataset is loaded and displayed to understand its structure, including values and labels.



Step 8 :- Data preview



Step 9 :- Train the Model

- Train using the dataset
- Model learns patterns from data here.
- The trained model is tested on new data to verify its performance. High accuracy ensures reliable predictions.

```
model.fit(X_train, y_train, epochs=20)

Epoch 1/20
Loss: 0.6923 - accuracy: 0.6285
Loss: 0.6283 - accuracy: 0.6285
Loss: 0.6171 - accuracy: 0.7685
Loss: 0.6123 - accuracy: 0.6909
Loss: 0.1918 - accuracy: 0.6100
Loss: 0.1125 - accuracy: 0.8900
```

Training the model with vibration data.

Step 10 :- Evaluate the Model

- Check the performance
- The model is tested on unseen data to check its performance. High accuracy indicates reliable fault detection.

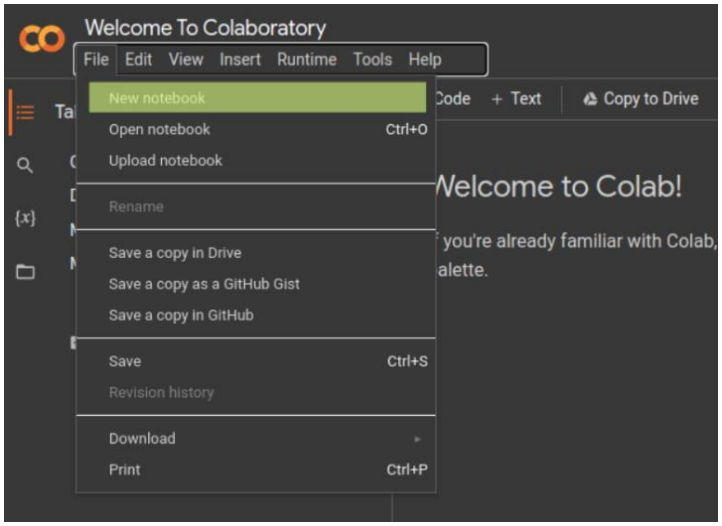
```
loss, acc = model.evaluate(X_test, y_test)
print('Accuracy:; acc)

Accuracy: 0.92
```

Evaluation result showing model accuracy.

Step 11 :- Saving Your Work

It's important to save your work periodically. You can save your notebook to Google Drive or download it to your computer as an .ipynb or .py file. Go to File > Save a copy in GitHub or File > Download.



4.5 MICRO PYTHON

Micro Python is a lightweight and efficient version of the Python specifically designed to run on microcontrollers such as the ESP32 and other embedded systems. It provides a simple and easy-to-use programming environment by using Python syntax, making it suitable for beginners as well as professionals. Micro Python includes only a small subset of the standard Python library, which allows it to operate on devices with limited memory and processing power. It enables direct interaction with hardware components like GPIO pins, sensors, and actuators, allowing users to control electronic devices efficiently. One of its key advantages is the ability to use an interactive interpreter (REPL), which helps in testing and debugging code in real time. Micro Python is widely used in applications such as Internet of Things (IoT), robotics, home automation, and embedded system projects. Overall, it simplifies hardware programming and speeds up development compared to traditional low-level languages like C.

CHAPTER-5

HARDWARE ASPECTS

5.1. POWER SUPPLY

A power supply is a fundamental unit in any electronic system that provides the necessary electrical energy for its operation. It acts as the source of power for all components and ensures that each part receives the required voltage and current in a controlled manner. Without a proper power supply, the system cannot function correctly.

The main function of a power supply is to convert the available input power into a stable and usable form. It maintains a constant output even when there are variations in input voltage or load conditions. This stability is important to prevent malfunction or damage to sensitive components.

A good power supply also includes filtering and regulation, which help in reducing noise and fluctuations in the output. This improves the overall performance, accuracy, and life of the system. Hence, the power supply plays a vital role in ensuring safe, efficient, and reliable operation of the entire project.

In this project, a regulated DC power supply is used to power the ESP32 and other components. The input supply is first converted and then regulated to maintain a constant voltage level. This helps to protect the system from voltage fluctuations and ensures reliable performance.

5.1.2 CLASSIFICATION OF POWER SUPPLY AND ITS DIFFERENT TYPES

Here we discuss different types of power supplies which have existed in the market world. The below table tells the basic types of power supplies for following conditions

TYPES OF SUPPLY	OUTPUT = DC	OUTPUT = AC
INPUT = AC	Bench power supplies Battery charger	Isolation transformer Variable AC supply
INPUT = DC	DC – DC Converter	UPS Inverter & Generator

Table 5.1.1: Types of Power Supply

5.2. ESP32

5.2.1. INTRODUCTION

The ESP32 is a highly advanced microcontroller designed for modern embedded systems and Internet of Things (IoT) applications. It is developed by Espressif Systems, which is well known for producing cost- effective wireless communication chips**.**

ESP32 is a complete system-on-chip (SoC) that integrates a microprocessor, memory, and wireless communication capabilities into a single compact unit. It is mainly used to develop smart devices that can sense, process, and communicate data efficiently. Due to its integrated design, it reduces the need for additional external components, making system design simpler and more economical.

The ESP32 is widely adopted in both academic and industrial fields because it provides a reliable platform for developing real-time applications. It supports interaction between hardware and software, allowing users to build systems that can monitor environmental conditions, control electronic devices, and exchange data over networks.

In recent years, ESP32 has become a key component in the growth of loT technology. It enables devices to be interconnected, forming intelligent systems that can operate automatically with minimal human intervention. This makes it suitable for applications such as smart homes, healthcare devices, industrial automation, and wireless communication systems.

Another important aspect of ESP32 is its flexibility in programming and development. It can be programmed using different platforms, which makes it accessible for beginners as well as professionals. Its compatibility with various sensors and modules allows developers to design a wide range of innovative projects.

Overall, the ESP32 serves as a powerful bridge between the physical world and digital systems. Its ability to combine processing and connectivity in a single device has made it one of the most widely used microcontrollers in the field of embedded systems and IoT development.

The ESP32 microcontroller is often mistaken for a chip for developers and enthusiasts only. What surprises most people is that many commercial smart home devices already include an ESP32 chip. Popular brands like Shelly, Wyze, Athom, and SONOFF offer products with Espressif's chip inside.

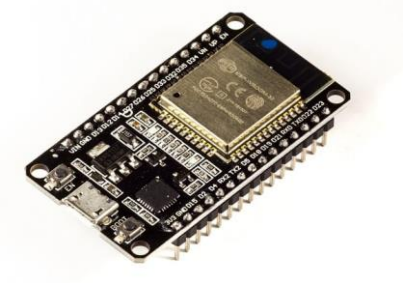


Figure 5.2.1: ESP32

5.2.2. PIN DESCRIPTION

The ESP32 pin diagram shows its flexibility, allowing it to interface with many devices and perform multiple functions, making it ideal for IoT and embedded projects.

- **Power Pins**

- VIN/3.3V → Supplies power to the board
- GND Ground connection

- **GPIO Pins (General Purpose Input/Output)**

- Used to connect LEDs, sensors, relays, etc.
- Example: GPIO 2, 4, 5, 18, 19, 21, 22, 23

- **ADC Pins (Analog Input)**

- Used to read analog signals
- Example: GPIO 32, 33, 34, 35

- **DAC Pins (Analog Output)**

- Used to generate analog signals
- Example: GPIO 25, 26

- **PWM Pins**

- Used for controlling brightness of LEDs and motor speed
- Most GPIO pins support PWM

- **Communication Pins**

- UART → TX (GPIO 1), RX (GPIO 3)
- I2C → SDA (GPIO 21), SCL (GPIO 22)
- SPI → MOSI, MISO, SCK pins

- **Touch Pins**

- Used for touch sensing applications
- Example: GPIO 4, 12, 13, 14

- **EN (Enable Pin)**

- Used to reset or enable the ESP32

5.2.3. PINOUT OF ESP32

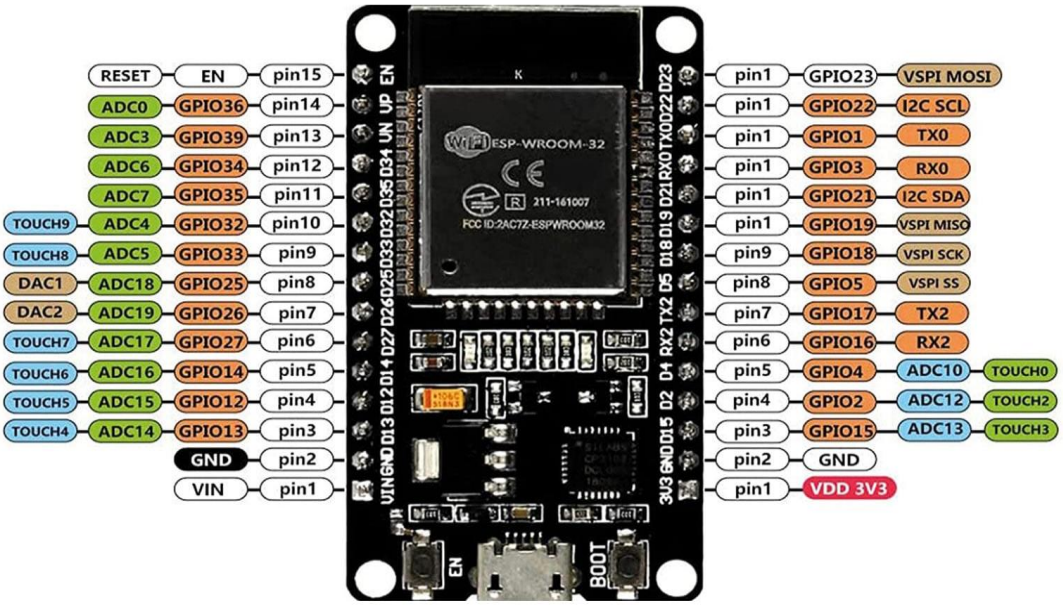


Figure 5.2.2: ESP32 PINOUT

5.2.4. FEATURES OF ESP32

- ESP32 contains a low-power Tensilica Xtensa® Dual-Core 32-bit LX6 microprocessor at 240 MHz:
- 994.26 CoreMark; 4.14 CoreMark/MHz
- 448 KB of ROM for booting and core functions.
- 520 KB of on-chip SRAM for data and instructions.
- 4MB of Flash Memory
- 16 KB SRAM in RTC
- Wi-Fi 802.11b/g/n
- Bluetooth v4.2 BR/EDR and Bluetooth LE specification

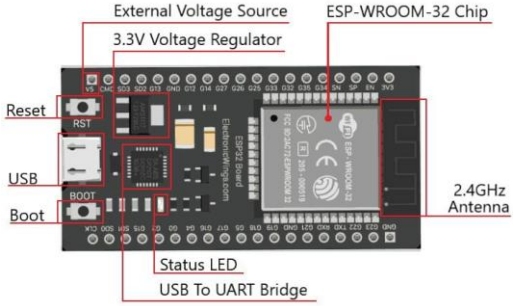


Figure 5.2.3: Features of ESP32

5.2.5. ADVANTAGES

The ESP32 offers several advantages that make it one of the most popular choices for embedded systems and loT projects.

- **Low Cost**
  - ESP32 is affordable while providing advanced features, making it suitable for students and large-scale projects.
- **High Performance**
  - It can handle complex tasks efficiently, making it ideal for real-time applications.
- **Built-in Connectivity**
  - Comes with integrated Wi-Fi and Bluetooth, reducing the need for external modules.
- **Low Power Consumption**
  - Supports power-saving modes, which is useful for battery-powered devices.
- **Versatile Usage**
  - Can be used in a wide range of applications like automation, robotics, and smart systems.
- **Compact Size**
  - Small in size, allowing easy integration into different electronic devices.
- **Strong Community Support**
  - Large number of tutorials, libraries, and resources are available for learning and development.
- **Easy Programming**
  - Supports platforms like Arduino IDE and Micro Python, making it beginner-friendly.
- **Multiple Interface Support**
  - Can connect with many sensors and devices easily.

5.2.6 APPLICATIONS

- **Smart Home Automation**
- Used to control lights, fans, appliances, and security systems remotely.
- **Industrial Automation**
- Helps in monitoring machines, controlling processes, and predictive maintenance systems.
- **IoT (Internet of Things) Systems**
- Used in devices that collect and share data over the internet, such as smart sensors and monitoring systems.
- **Wearable Devices**
- Applied in smartwatches, fitness trackers, and health monitoring devices**.**
- **Wireless Sensor Networks**
- Connects multiple sensors to monitor temperature, humidity, motion, etc
- **Robotics**
- Used for controlling robots and enabling wireless communication between components.
- **Smart Agriculture**
- Helps in irrigation control, soil monitoring, and crop management systems.
- **Healthcare Systems**
- Used in patient monitoring and remote health tracking devices.
- **Smart Cities**
- Applied in traffic management, smart lighting, and environmental monitoring.
- **Home Security Systems**
- Used in alarm systems, motion detection, and surveillance systems.

5.3 DHT11

5.3.1. INTRODUCTION

The DHT11 sensor is a basic and low-cost digital sensor used to measure both temperature and humidity of the surrounding environment. It is widely used in electronics projects, especially with microcontrollers like ESP32 and Arduino.

The DHT11 sensor contains a thermistor for measuring temperature and a capacitive humidity sensor for measuring moisture in the air. It also has an internal chip that converts these readings into digital signals, so it can easily communicate with microcontrollers using a single data pin.

This sensor can measure temperature in the range of 0°C to 50°C with an accuracy of about ±2°C, and humidity from 20% to 80% with an accuracy of ±5%. It provides output every 1 second, making it suitable for basic monitoring applications.

The DHT11 is commonly used in weather stations, home automation systems, air quality monitoring, and agricultural projects. Due to its simplicity and low cost, it is ideal for beginners, although it is less accurate compared to advanced sensors like DHT22 sensor.

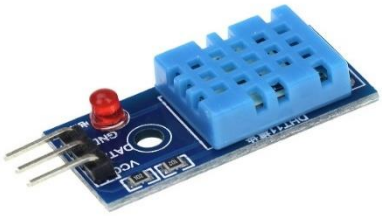


Figure 5.3.1: DHT11

5.3.2. DHT11 DESCRIPTION

Pin no	Pin name	Pin description
1	VCC	Power supply 3.3 to 5.5 Volt Dc
2	DATA	Digital output pin
3	GND	Ground

Table 5.3.1: DHT11 Description

5.3.3. DHT11 PINOUT

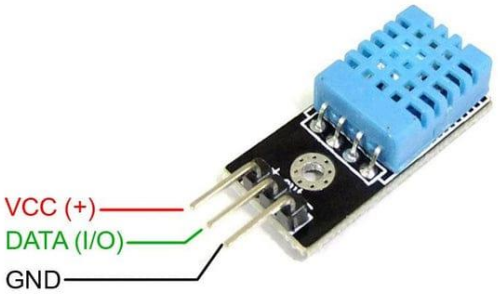


Figure 5.3.2: DHT11 Pinout

5.3.4. SPECIFICATION OF DHT11 SENSOR

- Power supply: 3.3 to 5V DC
- Current consumption: max 2.5mA
- Operating range: 20-80% RH, 0-50°C
- Humidity measurement range: 20-90% RH
- Humidity measurement accuracy: ±5% RH
- Temperature measurement range: 0-50°C
- Temperature measurement accuracy: ±2°C
- Response time: 1s
- Sampling rate: 1Hz (1 sample per second)
- Data output format: single-bus digital signal
- Data transmission distance: 20-30m (at open air)
- Dimensions: 15mm x 12mm x 5.5mm
- Weight: 2.5g

5.3.5. ADVANTAGES

- **Low cost - very economical and widely available**
- **Easy interfacing -** works easily with microcontrollers like ESP32 and Arduino
- **Digital output -** no need for analog-to-digital conversion
- **Pre-calibrated -** provides accurate readings without extra calibration
- **Low power consumption -** suitable for battery-operated devices

- **Compact size -** small and lightweight
- **Stable performance -** gives consistent readings over time
- **Simple wiring -** requires only a few connections

5.3.6. APPLICATIONS

The DHT11 is widely used in projects where temperature and humidity need to be monitored:

- **Weather monitoring systems** – used to measure environmental conditions
- **Home automation systems** – controls AC, fans, and humidifiers
- **Greenhouses and agriculture** – maintains proper humidity for plant growth
- **Industrial monitoring** – checks temperature and humidity in factories
- **IoT-based projects** – used with controllers like ESP32 for smart systems
- **HVAC systems** – helps in heating, ventilation, and air conditioning control

5.4 VIBRATION SENSOR

5.4.1. INTRODUCTION

A vibration sensor is an electronic device used to detect vibrations, shocks, and movements in objects or machines. It works by converting mechanical energy (vibrations) into electrical signals, which can be measured and analyzed. These sensors play an important role in monitoring the condition and performance of systems.

There are different types of vibration sensors such as piezoelectric sensors, accelerometers, and simple switch-based vibration modules. When a vibration occurs, the sensor generates a signal that can be read by a microcontroller like the ESP32 for further processing and decision-making.

Vibration sensors are widely used in industrial machines to detect faults, imbalance, or wear and tear. They are also used in security systems to detect tampering or intrusion, in automobiles for safety and monitoring, and in home appliances to detect abnormal movement. In IoT-based systems, vibration sensors help in predictive maintenance by identifying problems before equipment failure occurs.

Overall, vibration sensors are essential for improving safety, reducing maintenance costs, and ensuring the smooth operation of machines and devices.

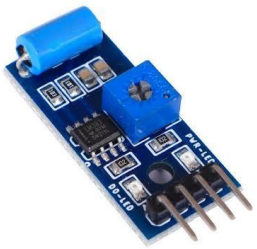


Figure 5.4.1: Vibration sensor

5.4.2. VIBRATION SENSOR PINOUT

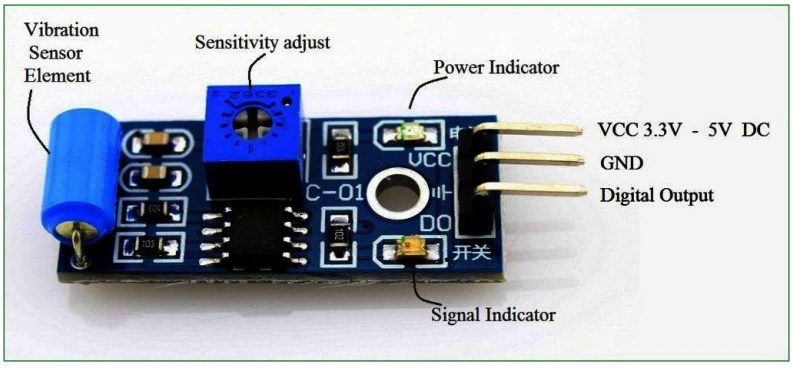


Figure 5.4.2: Vibration sensor pinout

5.4.3. SPECIFICATIONS OF VIBRATION SENSOR

The specifications of a vibration sensor may vary by type, but common specifications include:

- **Operating Voltage:** Typically 3.3V to 5V (suitable for microcontrollers like ESP32)
- **Output Type:** Digital (HIGH/LOW) or Analog signal
- **Sensitivity:** Adjustable (in some modules using a potentiometer)
- **Detection Range:** Depends on vibration intensity and sensor type
- **Response Time:** Fast response to small vibrations
- **Operating Temperature:** Usually -10°C to 60°C
- **Power Consumption:** Low power usage
- **Interface**: Simple connection (VCC, GND, Output pin)

5.4.4. ADVANTAGES

i. Detects vibrations and shocks quickly

ii. Helps in early fault detection in machines

iii. Improves safety and prevents damage

iv. Low cost and easy to use

v. Low power consumption

vi. Can be easily interfaced with controllers like ESP32

vii. Suitable for industrial and IoT applications

5.4.5 APPLICATIONS

- ❖ Machine condition monitoring
- ❖ Predictive maintenance systems
- ❖ Security and alarm systems (shock detection)
- ❖ Automobile monitoring systems
- ❖ Industrial equipment monitoring

5.5 PUMP

5.5.1. INTRODUCTION

A pump motor is an electrical device that drives a water pump by converting electrical energy into mechanical energy. It provides the necessary force to rotate the pump and move water from one place to another.

Pump motors are commonly electric motors (AC or DC) and are widely used in irrigation systems, domestic water supply, and industrial applications. They can also be controlled using microcontrollers like ESP32 in automated systems.

Pump motors are known for their efficiency, reliability, and continuous operation, making them essential in water pumping systems.

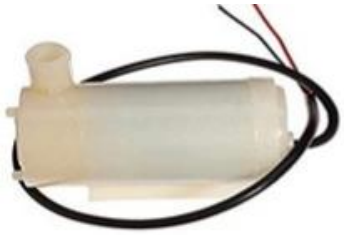


Figure 5.5.1: Pump

5.5.2 SPECIALIZATION OF PUMP

The specialization of a pump motor refers to its specific design and features that make it suitable for particular applications:

- **Type of Motor:** Can be AC or DC depending on the power source and application.
- **Power Rating:** Motors are available in different horsepower (HP) or wattage for small to large pumping needs.
- **Speed Control:** Some pump motors allow variable speed for precise water flow control.
- **Efficiency:** Designed to convert electrical energy to mechanical energy with minimal loss.
- **Durability**: Built to operate continuously in wet or harsh conditions.
- **Compatibility:** Can be coupled with different types of pumps such as centrifugal, submersible, or diaphragm pumps.

5.5.3 APPLICATIONS

- **Domestic Water Supply:** Provides water to homes from tanks or borewells
- **Agricultural Irrigation:** Powers pumps for fields, farms, and greenhouse
- **Industrial Water Systems:** Circulates water in factories, cooling systems, and processing units
- **Sewage and Drainage**: Pumps wastewater in drainage systems
- **Fountains and Landscaping:** Powers decorative water features and ponds
- **Automated Systems:** Works with microcontrollers like ESP32 for smart water management and automation

5.6. BUZZER

A buzzer is an electronic device that produces sound or alarm when electrical power is applied. It is widely used in electronic circuits to indicate events, warnings, or alerts.

- **Active Buzzer:** Produces sound when powered directly.
- **Passive Buzzer**: Requires an external signal (like a square wave) to produce sound.

Buzzers are commonly used in alarm systems, timers, electronic devices, and IoT projects with microcontrollers like ESP32 for alerts and notifications. They are small, low-cost, and easy to integrate into circuits.

Figure5.6.1: Buzzer

5.7. LED

A LED (Light Emitting Diode) is a small electronic component that emits light when electric current passes through it. It is energy-efficient, long-lasting, and commonly used as an indicator, in displays, and in circuits with controllers like ESP32.

Figure 5.7.1: Led

5.8. CONNECTING WIRES

Jumper wires are short, flexible wires used to connect different components in electronic circuits, especially on breadboards or between modules and microcontrollers like ESP32. They come in different types, such as male-to-male, male-to-female, and female-to-female, allowing easy connections without soldering. Jumper wires are widely used in prototyping, circuit testing, and electronics projects because they are reusable, flexible, and convenient for quickly setting up and modifying circuits.

Figure 5.8.1: Jumper Wires

CHAPTER -6
CIRCUIT DIAGRAM

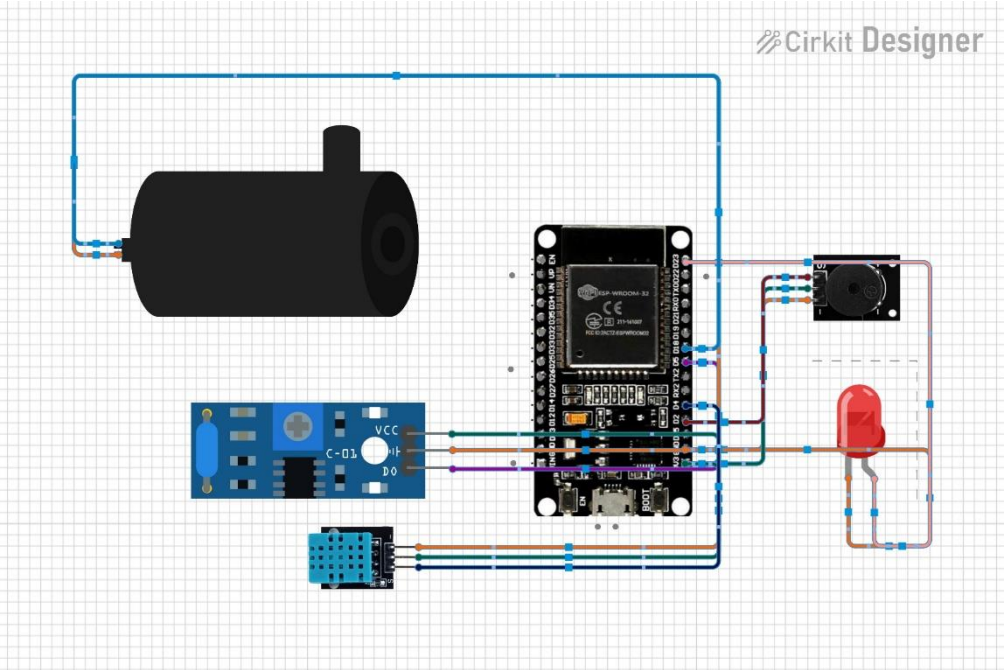


Figure 6.1: Circuit Diagram

6.1. CONNECTIONS

- **ESP32** is the main controller connected to all components.

- **DHT11 Sensor**
  - VCC → 3.3V
  - GND → GND
  - Data → GPIO 4
    → Measures temperature & humidity
- **Vibration Sensor**
  - VCC → 3.3V
  - GND → GND
  - DO → GPIO 5
    → Detects vibrations
- **LED**
  - Anode → GPIO 23
  - Cathode → GND
    → Indicates fault
- **Buzzer**
  - VCC → 3.3V
  - GND → GND
  - Signal → GPIO 2
    → Gives alert sound
- **Pump (Motor)**
  - Connected to power and controlled by ESP32

    → Turns ON during abnormal condition

- **All GNDs are common** and ESP32 controls everything based on sensor

CHAPTER-7

EXPERIMENT RESULT

7.1. RESULT

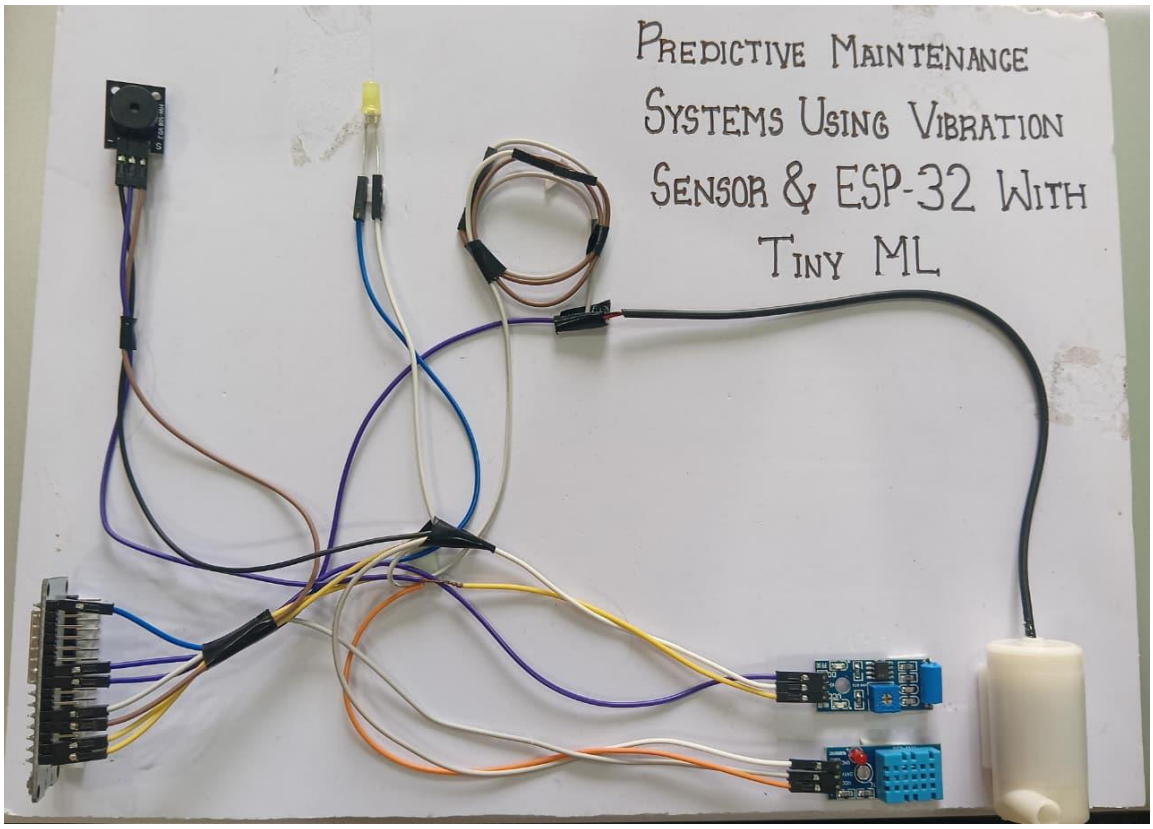


Figure 6.1. Result

The system successfully implemented smart predictive maintenance using a vibration sensor and ESP32 with Tiny ML. The vibration data was analyzed using a machine learning model to detect abnormal patterns in the pump. When unusual vibrations were detected, the system provided alerts through an LED and buzzer, helping to prevent potential pump failures. This approach improves maintenance efficiency, reduces downtime, and ensures reliable operation.

ADVANTAGES

1. **Early Fault Detection:** The vibration sensor detects unusual vibrations in machines, helping prevent sudden failures.
2. **Reduces Downtime:** Predictive alerts allow maintenance before breakdowns, minimizing production or operational interruptions.
3. **Energy Efficient:** Controlled use of the pump and other components reduces unnecessary power consumption.
4. **Automated Monitoring:** ESP32 enables real-time monitoring of machine health and environmental conditions (temperature and humidity via DHT11) without manual checks.
5. **Cost Savings:** Early detection of faults reduces repair costs and extends the life of machinery.
6. **Safety Improvement**: Alerts via LED and buzzer warn operators of abnormal conditions, enhancing workplace safety.
7. **Data-Driven Maintenance**: Continuous monitoring allows better planning of maintenance schedules based on actual machine condition**.**
8. **Integration with IoT:** ESP32 allows remote monitoring and potential integration with smart systems for better control.

APPLICATIONS

This project is useful anywhere continuous monitoring of equipment and early fault detection is important.

1. **Industrial Equipment Monitoring**: Detects faults in motors, pumps, and machines to prevent downtime.
2. **Manufacturing Plants**: Monitors machinery health and ensures smooth production processes.
3. **HVAC Systems**: Controls temperature and humidity while monitoring equipment for maintenance needs.
4. **Water Pump Systems**: Monitors pump performance and alerts for abnormal operation.
5. **IoT-Based Smart Systems:** Enables remote monitoring and predictive alerts for industrial or home automation.
6. **Safety Systems**: Alerts operators about equipment faults through LED and buzzer signals.
7. **Agricultural Automation**: Maintains optimal conditions for irrigation pumps and greenhouse systems.

CHAPTER-8

CONCLUSION AND SCOPE

CONCLUSION:

The smart predictive maintenance system using a vibration sensor and ESP32 with tiny ml provides a smart and efficient solution for monitoring machines and environmental conditions. It can detect early signs of faults through vibration, temperature, and humidity data, preventing unexpected breakdowns and reducing downtime and maintenance costs. The system's real-time alerts via LED and buzzer enhance safety and ensure timely action. This project highlights the importance of IoT-based monitoring, demonstrating how automation and sensor technology can improve equipment reliability, optimize operations, and make industrial, agricultural, and home systems more efficient and safe.

Future scope:

The smart predictive maintenance system using vibration sensor and ESP32 with tiny ml has a wide scope for future development. In the future, It can be expanded to monitor multiple machines or industrial equipment simultaneously, the system can be integrated with cloud connecting and IoT integration for real-time remote monitoring and data analysis. Advanced machine learning algorithms can be used to predict faults more accurately and schedule maintenance automatically. Mobile app or SMS alerts can notify operators instantly about abnormal conditions. The system can also be scaled to monitor multiple machines simultaneously and optimize energy usage for pumps and motors. Overall, with further enhancements, this project can evolve into a fully automated, smart, and efficient industrial maintenance solution.

APPENDIX

```
#include <DHT.h>
#include "dht_model.h"
#include "vibration_model.h"

Eloquent::ML::Port::dhtModel model;
Eloquent::ML::Port::vibrationModel vibrationmodel;

#define DHTPIN 4
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);
#define VIBRATION_PIN 5
#define LED_BUILTIN 23
#define BUZZER_PIN 2
#define PUMP_PIN 18

int SensorValue = 0;
//int lowlevel = 300;
//int highlevel = 700;

void setup() {
  Serial.begin(115200);
  Serial.println("Vibration Sensor Ready...");

  dht.begin();
  pinMode(VIBRATION_PIN, INPUT);
  pinMode(LED_BUILTIN, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(PUMP_PIN, OUTPUT);
  pinMode(DHTPIN, INPUT);

  digitalWrite(LED_BUILTIN, LOW);
  digitalWrite(PUMP_PIN, LOW);
}

void loop() {
  delay(2000);

  float humidity = dht.readHumidity();
  float temperature = dht.readTemperature();

  int vibration = digitalRead(VIBRATION_PIN);

  if(isnan(humidity) || isnan(temperature)){
    Serial.println(("Sensor Error!"));
    return;
  }
```

```
Serial.print("Temperature:");
Serial.print(temperature);
Serial.print("°C");

Serial.print("Humidity:");
Serial.print(humidity);
Serial.println("%");

Serial.println("------------------");
if(vibration == HIGH ){
  Serial.println("Vibration Detected!");
  digitalWrite(PUMP_PIN,HIGH);
  digitalWrite(LED_BUILTIN, HIGH );
  delay(200);
  digitalWrite(LED_BUILTIN, LOW );
  delay(20);
  digitalWrite(LED_BUILTIN, HIGH );
  tone(BUZZER_PIN, 2000);
}
else{
  Serial.println("No Vibration");
  digitalWrite(LED_BUILTIN,LOW );
  digitalWrite(PUMP_PIN, LOW);
  noTone(BUZZER_PIN);

}
float features1[3];
float features2[1];

features1[0] = digitalRead(4);
features2[0] = digitalRead(5);

int prediction1 = model.predict(features1);
int prediction2 = model.predict(features2);
int prediction3 = 0;

Serial.print("Sensor Value 1: ");
Serial.println(features1[0]);

Serial.print("Prediction 1: ");
Serial.println(prediction1);

Serial.print("Sensor Value 2: ");
Serial.println(features2[0]);

Serial.print("Prediction 2: ");
Serial.print(prediction2);

if (prediction1 == 1 && prediction2 == 0){
 Serial.println("Fault Detect");
```

```
  } else {
   Serial.println("No Fault Detect");
  }
  SensorValue = digitalRead(18);
Serial.println(SensorValue);
  if (prediction3 == 1){
    digitalWrite(PUMP_PIN, HIGH);
  }
  else {
    digitalWrite(PUMP_PIN, LOW);
  }
}
```

.h Files Codes

For vibration.h

```
#pragma once
#include <cstdarg>
namespace Eloquent {
    namespace ML {
        namespace Port {
            class DecisionTree {
                public:
                    /**
                    * Predict class for features vector
                    */
                    int predict(float *x) {
                        if (x[0] <= 0.5) {
                            return 0;
                        }

                        else {
                            return 1;
                        }
                    }

                protected:
                };
            }
        }
    }
```

For Dht.h

```
#pragma once
#include <cstdarg>
namespace Eloquent {
    namespace ML {
        namespace Port {
            class DecisionTree {
                public:
                    /**
                    * Predict class for features vector
                    */
                    int predict(float *x) {
                        if (x[0] <= 80.04999923706055) {
                            return 1;
                        }

                        else {
                            return 0;
                        }
                    }

                protected:
                };
            }
        }
    }
```

REFERENCE

1. **Gupta, S., & Shivhare, S. N. (2025).**
"Embedded TinyML for Predictive Maintenance: Vibration Analysis on ESP32 with Real-Time Fault Detection in Industrial Equipment." International Journal on Computational Modeling Applications.

2. **Pathak, Y., Dubey, M., & Sharma, T. (2026).**
"Confidence-Aware Tiny Machine Learning for Vibration-Based Predictive Maintenance in Automotive Systems." Research square.

3. **Vasilache, A., et al. (2024**).
"Low-Power Vibration-Based Predictive Maintenance Using Neural Networks: A Survey."
arXiv.

4. **Samatas, G. G., Moumgiakmas, S. S., & Papakostas, G. A. (2021).**
"Predictive Maintenance: Bridging Artificial Intelligence and IoT." arXiv.

5. **Ong, K. S. H., Niyato, D., & Yuen, C. (2020).**
"Predictive Maintenance for Edge-Based Sensor Networks Using Deep Reinforcement Learning." arXiv.

6. **Malawade, A. V., et al. (2021).**
"Neuroscience-Inspired Algorithms for Predictive Maintenance of Manufacturing Systems."

7. **Chen, H. Y., & Lee, C. H. (2020).**
"Vibration Signal Analysis Using Explainable AI for Bearing Fault Diagnosis."
IEEE Access.

8. **Zhang, W., Yang, D., & Wang, H. (2019).**
"Data-Driven Methods for Predictive Maintenance of Industrial Equipment."
IEEE Systems Journal.

9. **Jardine, A. K. S., Lin, D., & Banjevic, D. (2006).**
"A Review on Machinery Diagnostics and Prognostics Implementing Condition-Based Maintenance."